

Algorithms

- properties
- basic blocks
- arithmetic operators

General considerations

- identify the necessary data structure
- a few extra/saved variables do not make a difference
- one relevant aspect is the number of statements that are executed
- sorting is a task that requires efforts

Exercises in class

Least frequent digit in number

Given an integer number, find the digit that occurs the fewest times among the present digits. If several digits share that minimum count, return the largest of them.

Version A solving strategy: Keep a counter for each possible digit in the number. For every digit in the input number, increment the corresponding counter. Starting from the counters of 9s, find the counter with the smallest count.

```
1 initialise 10 counters to 0
2 acquire num
3 for every digit in num
4     increment corresponding counter
5     start from counter of 9s
6 while the counter value is 0
7     go to the next counter
8 min_occ = value of the counter
9 min_digit = digit of the counter
10 if min_occ is greater than 1
11     for all remaining counters
12         if the counter value is greater than 0 and smaller than min_occ
13             min_occ = value of the counter
14             min_digit = digit of the counter
15             if min_occ is 1, interrupt iterations
16 display min_digit
```

- Data structure: ten counters
- Input number analysis (digit extraction): only once
- Computation: the digits of the input number are extracted once, and the counters are scanned once
- Optimization: analysis interrupted as soon as result is found

Version B solving strategy: count how many times each possible digit appears in the input number and keep track of the smallest number of occurrences greater than 0 and corresponding to the highest digit.

```
1 acquire num
2 for every value v from 9 to 0
3     count how many times it appears in num
4     if it is greater than 0
5         min_occ = number of times
6         min_digit = value
7         interrupt iterations
```

```

8 if min_occ greater than 1
9     for every value v from min_digit-1 to 0
10         count how many times it appears in num
11         if it is greater than 0 and smaller than min_occ
12             min_occ = number of times
13             min_digit = value
14             if min_occ is 1, interrupt iterations
15 display min_digit

```

- Data structure: none
- Input number analysis (digit extraction): performed ten times
- Computation: the input number digits are extracted ten times
- Optimization: analysis interrupted as soon as result is found

Version C solving strategy: for every digit in the input number, if we have not already found it, count how many times it appears in the number and keep track of the smallest number of occurrences and corresponding to the highest digit

```

1 list of digit_list set to empty
2 acquire num
3 for each digit in num
4     if it is not in digit_list
5         add it in a descending sorted way in digit_list
6 take the first digit d in digit_list
7 count how many times it appears in num: min_occ
8 min_digit = d
9 if min_occ greater than 1
10     for every remaining digit d in digit_list
11         count how many times it appears in num: num_occ
12         if it is smaller than min_occ
13             min_occ = num_occ
14             min_digit = d
15             if min_occ is 1, interrupt iterations
16 display min_digit

```

- Data structure: list
- Input number analysis (digit extraction): one to scan of the input and one for every different digit in the input number to count occurrences
- Computation: the created list is scanned once and for each unique digit, the number of occurrences is counted (the remaining part of the input number is analysed)
- Optimization: analysis interrupted as soon as result is found

Version D solving strategy: build a list of unique digits appearing in the input number, sorted in a descending order. For each digit count how many times it appears in the input number and find the smallest number of occurrences.

```

1 list of digit_list set to empty
2 acquire num
3 for each digit in num
4     if it is not in digit_list
5         add it in a descending sorted way in digit_list
6 take the first digit d in digit_list
7 count how many times it appears in num: min_occ
8 min_digit = d
9 if min_occ > 1
10     for every remaining digit d in digit_list
11         count how many times it appears in num: num_occ
12         if num_occ is smaller than min_occ

```

```

13         min_occ = num_occ
14         min_digit = d
15         if min_occ is 1, interrupt iterations
16 display min_digit

```

- Data structure: list, maintained sorted and with no duplicates
- Input number analysis (digit extraction): one scan of the input and one for every different digit in the input number to count occurrences
- Computation: the created list is scanned once and for each unique digit, the number of occurrences is counted (input number is analysed)
- Optimization: analysis interrupted as soon as result is found

Version E solving strategy: Extract all digits and insert them into a descending sorted list. Measure each subsequence of identical elements length and find the shortest one.

```

1 list of digit_list set to empty
2 acquire num
3 for each digit in num
4     add it in a descending sorted way in digit_list
5 start from the beginning of the list
6 set the length of the subsequence of identical elements to 1
7 while there is a subsequent element and it is equal to the initial one
8     increment the length of the subsequence
9     move to the next element in the list
10 min_occ = length of the subsequence
11 min_digit = first element of the list
12 move to the first element of the next subsequence
13 while min_occ > 1 and there are elements remaining
14     curr_dig = current digit in the list
15     set the length of the subsequence of identical elements to 1
16     while there is a subsequent element in the list and it is equal to curr_dig
17         increment the length of the subsequence
18         move to the next element in the list
19     if length of the subsequence is smaller than min_occ
20         min_occ = num_occ
21         min_digit = curr_dig
22         if min_occ is 1, interrupt iterations
23     move to the first element of the next subsequence
24 display min_digit

```

- Data structure: list, maintained sorted
- Input number analysis (digit extraction): input number is scanned only once
- Computation: the created list is scanned once to compute subsequence of identical adjacent elements
- Optimization: analysis interrupted as soon as result is found

Simulation pages

- version A
- version B
- version C
- version D
- version E

Anagrams

Given two sequences of characters, determine if they are one the anagram of the other.

ATM

Given a withdrawal amount that is a multiple of €5, determine the minimum number of banknotes required using €50, €20, €10 and €5 notes, and display the quantity of each denomination.

Solving strategy: Use as many €50 bills as possible, then cover the remaining amount with as many €20 bills as possible, cover the remaining amount with as many €10 bills as possible, and finally use €5 bills.

```
1 acquire amount
2 n50 = integer (amount / 50)
3 rem = amount - n50 * 50
4 n20 = integer (rem / 20)
5 rem = rem - n20 * 20
6 n10 = integer (rem / 10)
7 n5 = (rem - n10 * 10) / 5
8 display n50, n20, n10, n5
```

using operators from python

```
1 acquire amount
2 n50 = amount // 50
3 rem = amount % 50
4 n20 = rem // 20
5 rem = rem % 20
6 n10 = rem // 10
7 n5 = (rem % 10) // 5
8 display n50, n20, n10, n5
```

Introducing an additional control to avoid the division requires the evaluation of an additional condition and other assignments.

```
1 acquire amount
2 n50 = amount // 50
3 rem = amount % 50
4 if rem greater than 0
5     n20 = rem // 20
6     rem = rem % 20
7     if rem greater than 0
8         n10 = rem // 10
9         n5 = (rem % 10) // 5
10    otherwise
11        n10 = 0
12        n5 = 0
13 otherwise
14     n20 = 0
15     n10 = 0
16     n5 = 0
17 display n50, n20, n10, n5
```

Proposed exercises

Identify an algorithm that asks for a positive number of seconds, then converts it into hours, minutes, and seconds, and finally displays the result.

Examples:

```
1 26 -> 0h 0m 26s -> 0 0 26
2 234 -> 0h 3m 54s -> 0 3 54
3 3725 -> 1h 2m 5s -> 1 2 5
```

Identify an algorithm that asks for a positive number of seconds, then converts it into days, hours, minutes, and seconds, and finally displays the result.

Examples:

```
1 26 -> 0h 0m 26s -> 0 0 26
2 234 -> 0h 3m 54s -> 0 3 54
3 3725 -> 1h 2m 5s -> 1 2 5
4 93789 -> 1d 2h 3m 9s -> 1 2 3 9
```

Identify an algorithm that acquires a positive integer value (it is surely so) representing the side of a square and computes the area and displays it.

Identify an algorithm that acquires a positive integer value (it is surely so) representing the radius of a circle and computes the length of the circumference and area, then displays the two values.

Identify an algorithm that acquires a positive integer value in input and calculates and displays 1 if the value is odd, or 0 otherwise. Make a variation that does not use the remainder operator.

Identify an algorithm that acquires a positive integer value (it is surely so) representing a year and it computes and displays 1 if it is a leap year, 0 otherwise (taken from Wikipedia, that is, commonly understood by the general public “this extra leap day occurs in each year that is a multiple of 4, except for years evenly divisible by 100 but not by 400.”)